

Lab 1: Identify Entities and Attributes from a Flat File

Problem Statement

You are given a CSV export from an e-commerce app that mixes order, customer, and product details in each row. Identify the candidate entities and list their key attributes.

Scenario / Input

Sample CSV rows (columns):

order_id, order_date,

customer_name, customer_email,

product_id, product_name, quantity, price, shipping_address

Entity	Attributes
order	order_id PK, order_date, total, shipping_address, customer_id FK
customer	customer_id PK, customer_name, customer_email, address, city, country
product	product_id PK, product_name, price, category
order item	order_itemID PK, quantity, product_id, order_id FK

Lab 2: Classify Relationships and Keys

Problem Statement

Using entities from Lab 1, state relationship types and propose PK/FK for each table. Include examples where orders contain multiple products.

Entity	PK	FK	Relationship notes
order	order_id	customer_id	One order corresponds to one customer. One order can contain multiple order_items (products).
customer	customer_id		One customer can place many orders.
product	product_id		The same product can be listed many times as an order item.
order item	order_itemID	order_id, product_id	Each order_item relates to one product in each order.

Lab 3: Apply First Normal Form (1NF)

Problem Statement

Transform the following denormalized row into 1NF rows.

Scenario / Input

One row:

```
order_id: 1002 customer: Bob Smith products: [ {"product_id": 1, "qty": 3}, {"product_id": 1, "qty": 2} ]
```

order_id	customer_name	products_id	qty
1002	Bob Smith	1	3
1002	Bob Smith	1	2

Lab 4: Convert Repeating Groups into Rows

Given a small JSON list of orders with product arrays, flatten into OrderItem rows using Python.

```
orders = [
    {"order_id": 1003, "customer": "Ana", "items": [{"product_id": 301, "qty": 2}],
    {"order_id": 1004, "customer": "Ben", "items": [{"product_id": 302, "qty": 1}, {"product_id": 303, "qty": 4}]}
]
```

//1- create an empty array/ list

```
order_items = [ ]
```

//2- use a for/in loop to iterate through every item in orders

for order in orders:

```
    for item in order["items"]:
```

```
        //add a new row to the list/ array
```

```
        order_items.append({
            "order_id": order["order_id"],
            "customer": order["customer"],
            "product_id": item["product_id"],
            "qty": item["qty"]
        })
```

```
print(order_items)
```

Lab 5: Apply Second Normal Form (2NF)

Problem Statement

You have a table OrderItem(order_id, product_id, product_name, product_price, qty). Identify partial dependencies and split tables to reach 2NF.

Table Product

product_id PK | product_name | product_price

Table OrderItem

order_itemId PK | order_id FK | product_id FK | qty

Lab 6: Apply Third Normal Form (3NF)

Problem Statement

Given Customer(customer_id, name, city, zip, state_name), identify transitive dependencies and suggest table splits to reach 3NF.

customer_id	name	city	zip	state_name
1	Ana	Beverly Hills	90210	California
2	Ben	Beverly Hills	90210	California

note: A **transitive dependency** is when a **non-key attribute depends on another non-key attribute rather than directly on the primary key**. Removing these dependencies is the goal of **Third Normal Form (3NF)**.

table 1

customer_id	name	city	zip
1	Ana	Beverly Hills	90210
2	Ben	Beverly Hills	90210

table 2

zip	city	state_name
90210	Beverly Hills	California

Lab 7: Design an OLTP Normalized Schema (Orders)

Problem Statement

Design a normalized OLTP schema for customers, orders, order_items, and products. Provide CREATE TABLE statements in SQL (simplified) with PK and FK constraints.

Scenario / Input

Assume typical e-commerce needs: enforce data integrity, avoid redundancy.

Try It Yourself

Write CREATE TABLE statements using basic types.

Hint

Use surrogate integer keys for PKs; show foreign keys.

```
CREATE TABLE Customer (  
  customer_id SERIAL PRIMARY KEY,  
  name VARCHAR(200),  
  email VARCHAR(200) UNIQUE,  
  default_shipping_address TEXT  
);
```

```
CREATE TABLE Product (  
  product_id SERIAL PRIMARY KEY,  
  product_name VARCHAR(200),  
  price NUMERIC(10,2)  
);
```

```
CREATE TABLE "Order" (  
  order_id SERIAL PRIMARY KEY,  
  customer_id INT REFERENCES Customer(customer_id),  
  order_date DATE,  
  shipping_address TEXT  
);
```

```
CREATE TABLE OrderItem (  
  order_item_id SERIAL PRIMARY KEY,  
  order_id INT REFERENCES "Order"(order_id),  
  product_id INT REFERENCES Product(product_id),  
  qty INT,  
  line_price NUMERIC(10,2)  
);
```

Lab 8: Create a Denormalized Reporting Table

Problem Statement

Create a denormalized daily orders reporting table that contains customer_name, order_id, order_date, total_order_value, product_list (as a single string), and shipping_city.

Scenario / Input

Sample normalized data (you can assume aggregation is needed).

Expected Output Format

Show CREATE TABLE for reporting and one sample INSERT/row produced by aggregation.

Try It Yourself

Design the reporting table and show an example aggregated row.

Hint

Product_list can be a comma-separated string for quick dashboards.

reporting table:

```
CREATE TABLE dailyOrderReport (  
  report_id SERIAL PRIMARY KEY,  
  order_id INT,  
  order_date DATE,  
  total_order_value DECIMAL(10,2)  
  product_list VARCHAR(500)  
  customer_name VARCHAR(200),  
  shipping_city VARCHAR(100)  
);
```

sample:

```
report_id: 42  
order_id: 457  
order_date: 2026-07-15  
total_order_value: 107.34  
product_list: Keyboard R16  
customer_name: John Doe  
shipping_city: London
```

A denormalized table stores extra or repeated data so it can be read faster.

The product_list is a ready-made list of products so reports and dashboards don't have to build the list every time -> faster readings.

Lab 9: Design a Star Schema: Fact and Dimension Tables

Problem Statement

Design a star schema for order analytics: FactOrders and dimensions CustomerDim, ProductDim, DateDim. Define grain, keys, and 5 sample columns for each.

Scenario / Input

Analytics use cases: daily sales by product category and customer region.

Expected Output Format

Table list with columns and brief explanation of grain.

Try It Yourself

Define primary keys for dims and foreign keys in the fact.

Hint

Fact grain: one row per order_item or per order (decide and justify).

Table	Key and sample columns
FactOrders	order_item_key PK, customer_key FK, product_key FK, date_key FK, order_id, quantity, unit_price, total_amount
CustomerDim	customer_key PK, customer_id, customer_name, customer_email, city, region
ProductDim	product_key PK, product_id, product_name, category, brand, current_price
DateDim	date_key PK, full_date, day, month, quarter, year

relationships:

- FactOrders.customer_key → CustomerDim.customer_key
- FactOrders.product_key → ProductDim.product_key
- FactOrders.date_key → DateDim.date_key

grain explanation

Each row in FactOrders represents one product within one order.

For example, if order 1 contains two products, the fact table will contain two rows.

Lab 10: Map Normalized Data to Star Schema (ETL sketch)

Level

Intermediate

Concepts Covered

- ETL mapping
- Data transformation for analytics

Problem Statement

Sketch SQL/ pseudocode to load fact table from normalized OLTP tables: orders, order_items, products, customers. Include joins and aggregations for daily fact (one row per order_item with sales_amount).

Scenario / Input

Normalized tables available in the source schema.

Expected Output Format

SQL query or pseudocode that produces fact rows.

Try It Yourself

Write an INSERT INTO ... SELECT ... statement.

Hint

Compute sales_amount = qty * product_price.

Lab 11: Model Slowly Changing Dimension (Type 2)

Level

Advanced

Concepts Covered

- Slowly Changing Dimensions (SCD)
- Type 2 implementation

Problem Statement

Design a CustomerDim that captures address history using Type 2 SCD. Show table columns and example rows demonstrating a customer moving city.

Scenario / Input

Customer 900 moved from "Oldtown" to "Newcity" on 2026-05-10.

Expected Output Format

Table definition and two sample rows showing versioning/history.

Try It Yourself

Decide on surrogate key, natural key, effective_from, effective_to, and current_flag.

Hint

Type 2 keeps history by inserting new rows, not updating.

Lab 12: Design for Query Performance: Denormalization Trade-offs

Level

Advanced

Concepts Covered

- Denormalization for performance
- Trade-offs (storage, updates, consistency)

Problem Statement

Given slow join-heavy reports, propose three denormalization changes to speed up queries and list the trade-offs for each.

Scenario / Input

Reports frequently join orders -> customers -> addresses -> product -> category.

Expected Output Format

List of 3 denormalization proposals with expected benefit and trade-off.

Try It Yourself

Rank proposals by read performance improvement.

Hint

Consider precomputed aggregates, wide reporting tables, and adding redundant columns.